

Adapter

Motivation

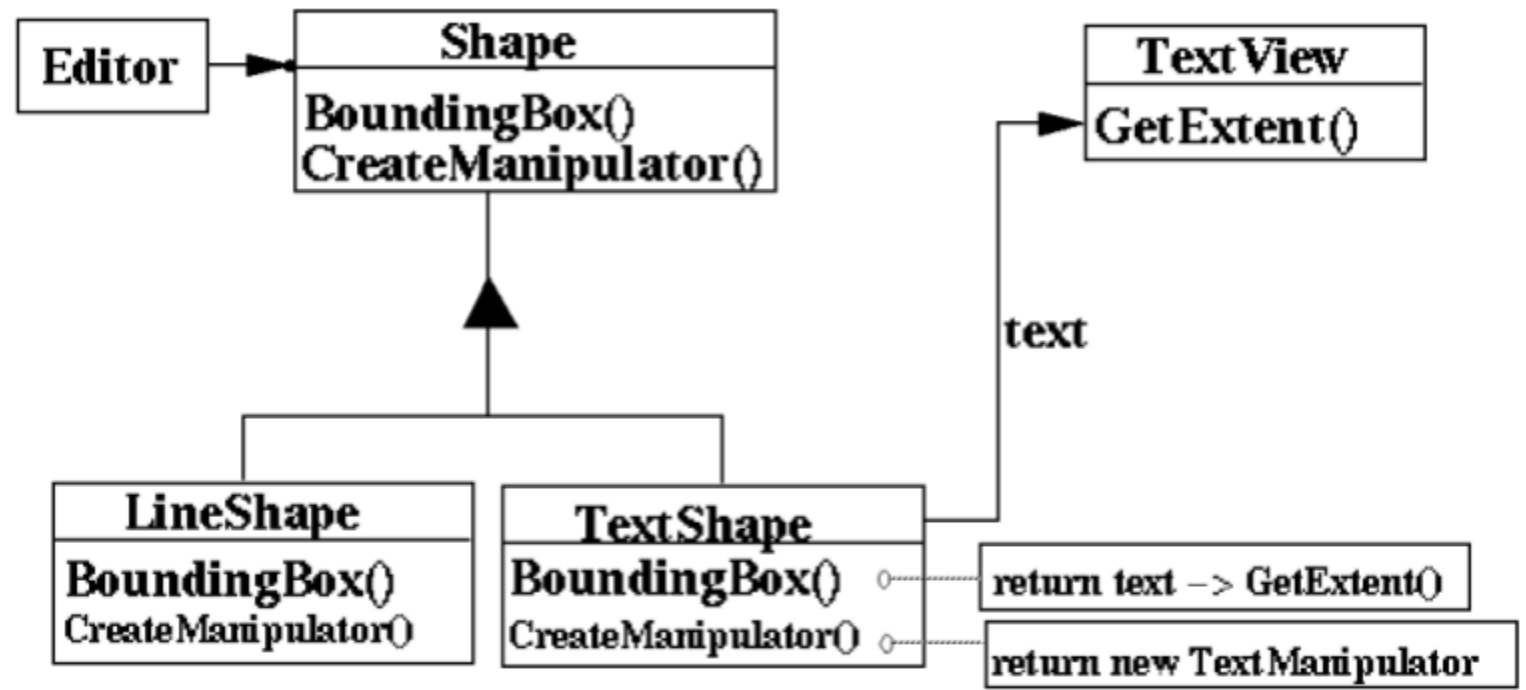
- When we want to reuse classes in an application that expects a different interface, we do not want to (and often cannot) change the reusable classes to suit our application

Intent

- Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces

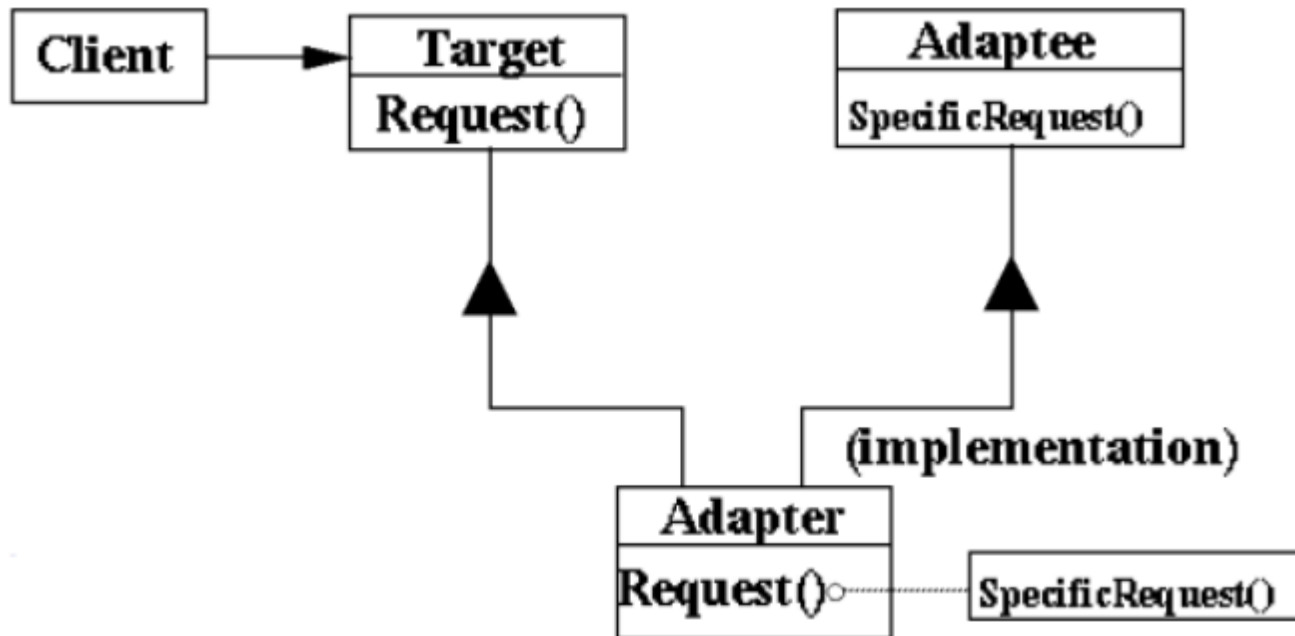
Shape Sample

- Structure -Example



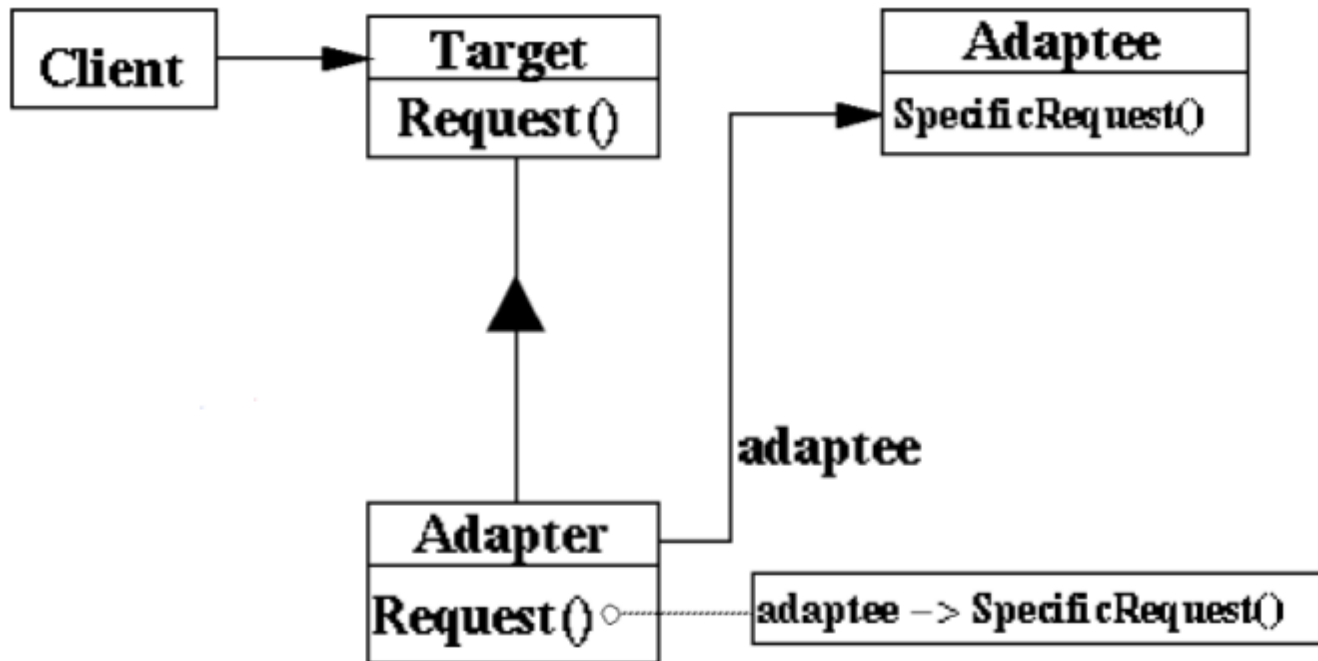
Adapter

- Structure with multiple inheritance(Class adapter)



Adapter

- Structure using object composition(Object Adapter)



Participants

- Target: Defines the application-specific interface that clients use
- Client: Collaborates with objects conforming to the target interface
- Adaptee: Defines an existing interface that needs adapting
- Adapter: Adapts the interface of the adaptee to the target interface

Class Adapter vs. Object Adapter

- Class adapter
 - ✓ (+?) Lets adapter override some of adaptee's behavior, since adapter is a subclass of adaptee
 - ✓ (+) Introduces only one object, and no additional pointer indirection is needed
 - ✓ (-) Adapts adaptee to target by committing to a concrete adaptee class. As a consequence, a class adapter won't work when we want to adapt a class and all its subclass.

Class adapter vs object adapter

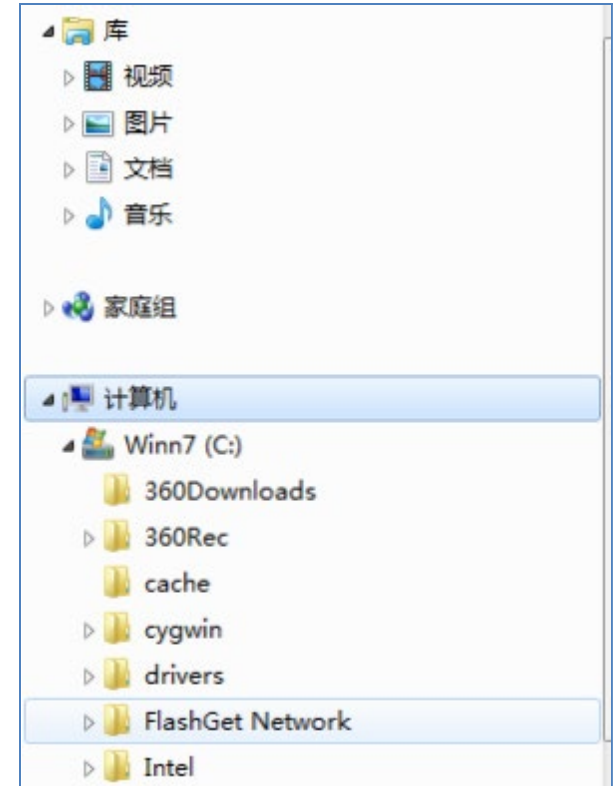
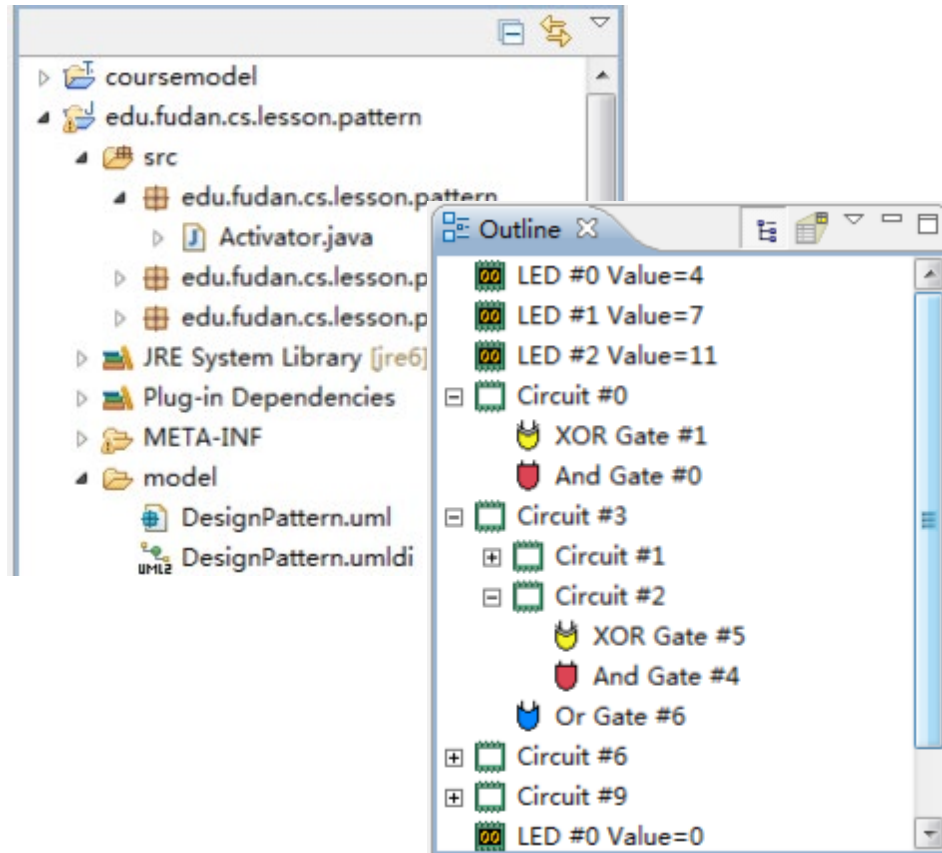
- Object adapter
 - ✓ (+) Lets a single adapter work with many kind of adaptees— that is, the adaptee itself and all of its subclasses.

Consequences

- How much adapting does Adapter do?
 - From changing the name of an operation to supporting an entirely different set of operations.
- Pluggable adapters
 - A class is more reusable when you minimize the assumptions other classes must make to use it. By building interface adaption into a class you eliminate the assumption that other classes see the same interface.

Sample 1:

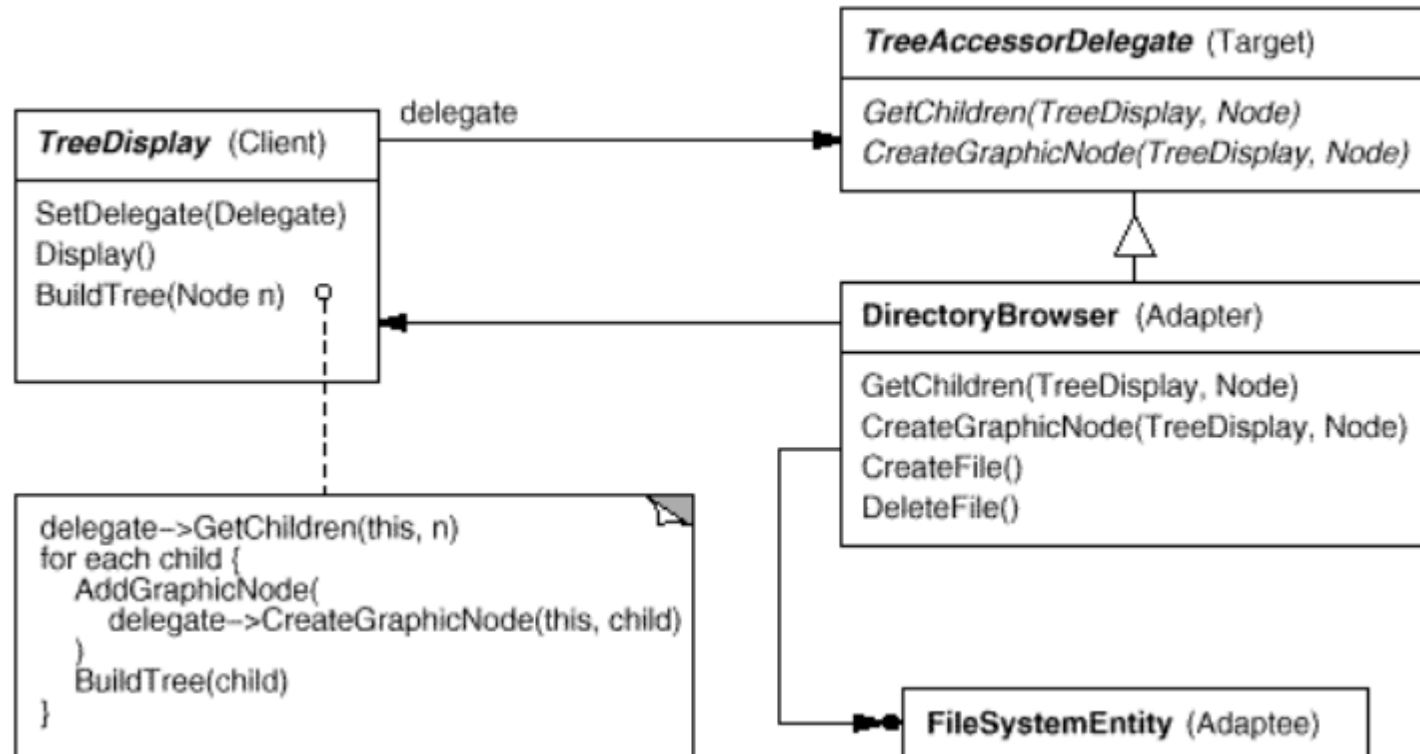
How to design a tree viewer widget



Sample 1:

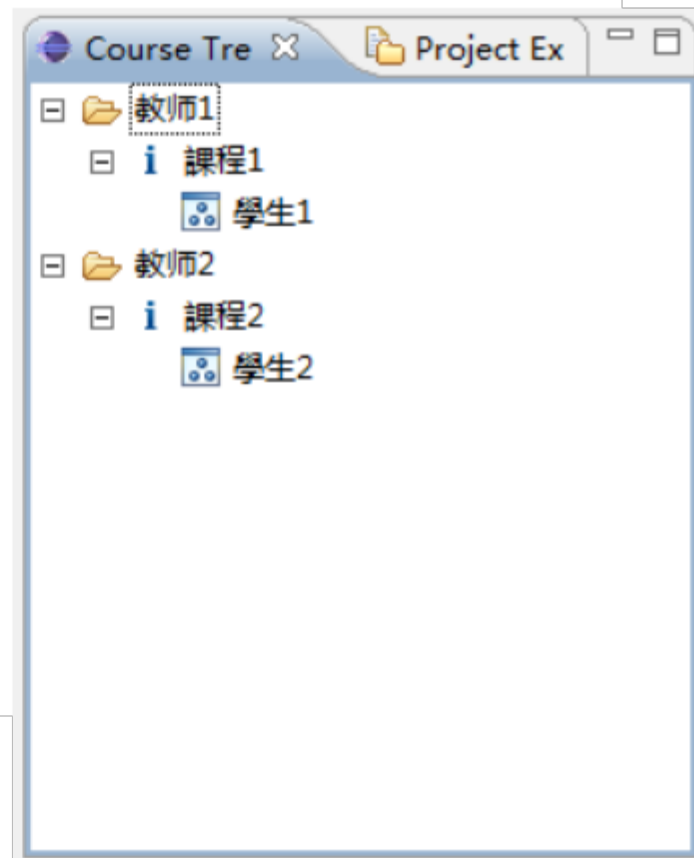
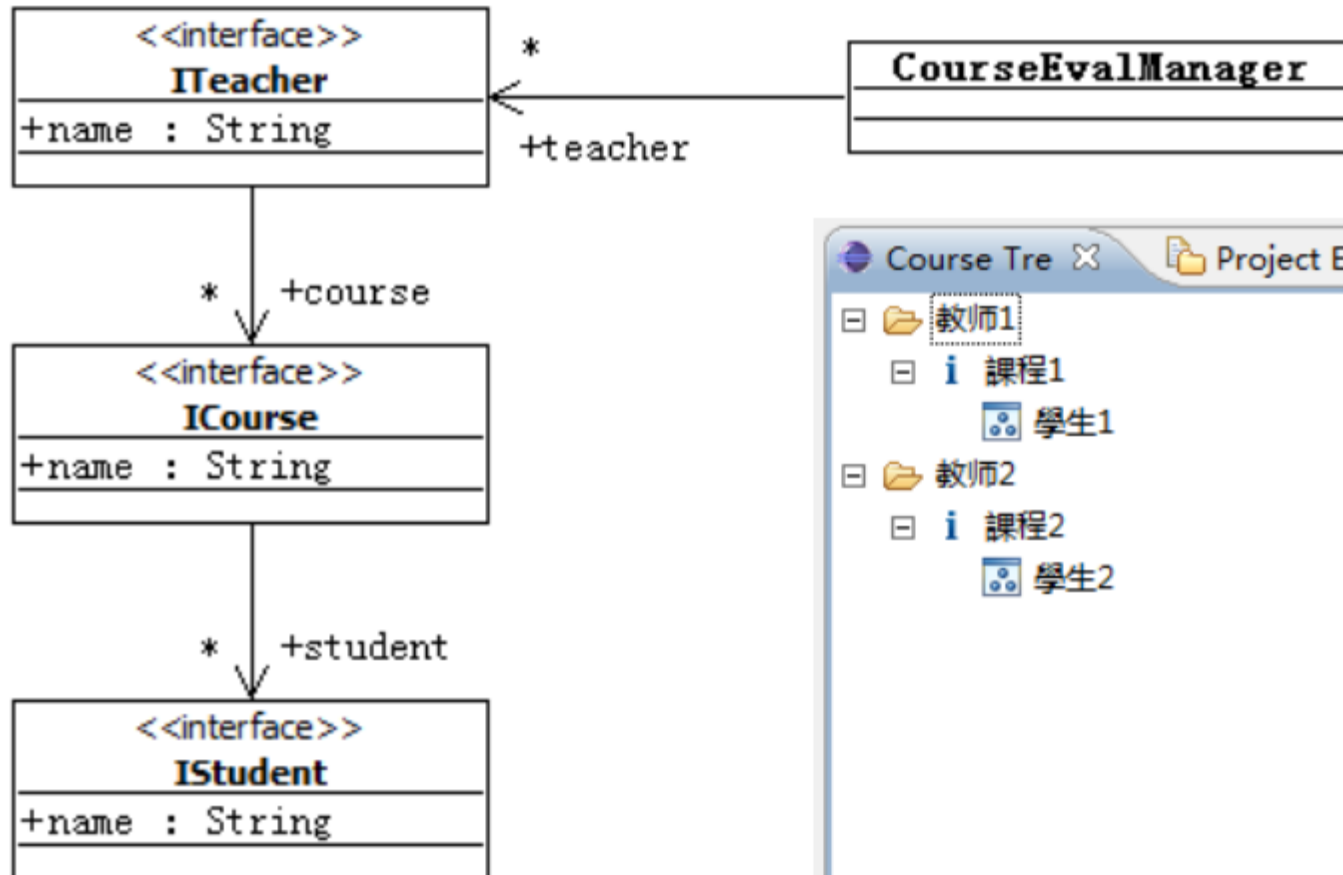
How to design a tree viewer widget

- Using delegate objects

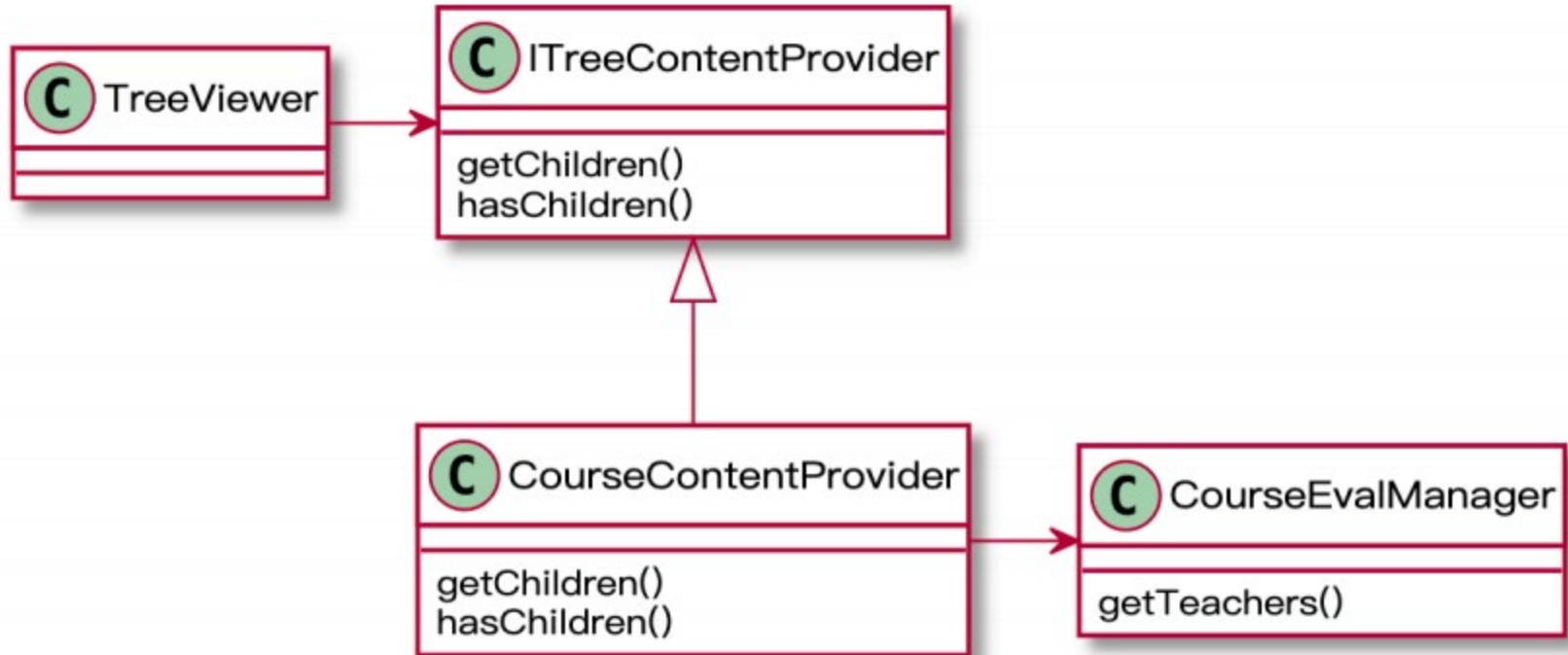


Sample:

How to design a tree viewer widget



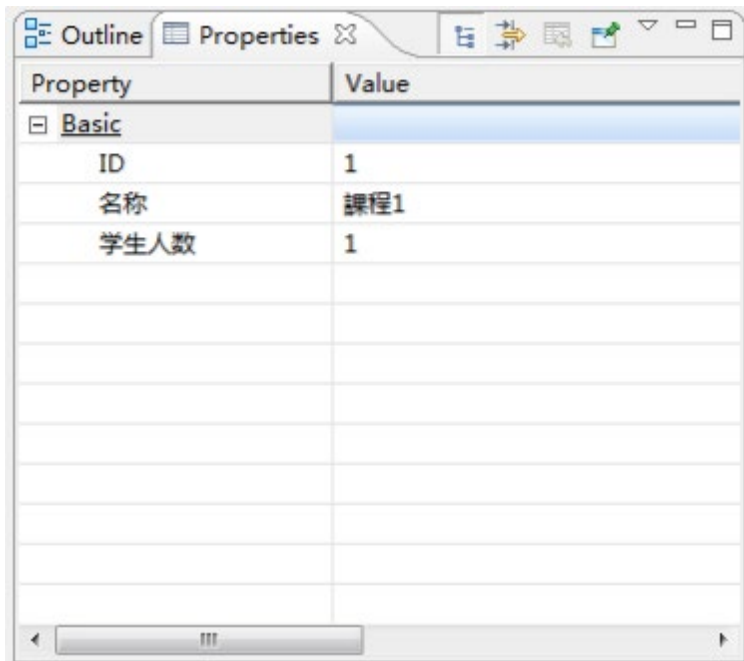
Implementation in Eclipse



```
ICourseEvaluationManager courseEvalManager = ....;  
viewer = new TreeViewer(parent,....);  
viewer.setContentProvider(new CourseContentProvider(courseEvalManager));
```

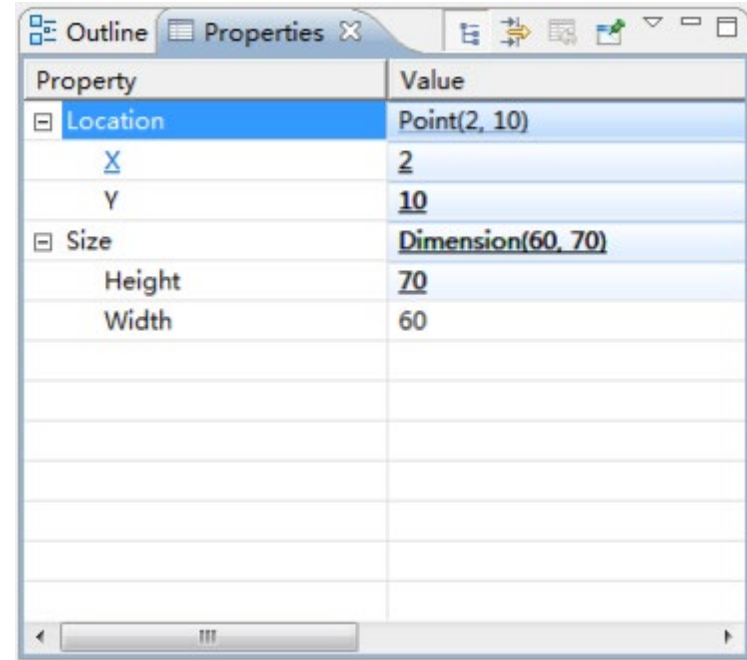
Sample:

Property view for course



This screenshot shows the 'Basic' property view in a software interface. The 'Outline' tab is active, and the 'Properties' pane displays a table with two columns: 'Property' and 'Value'. The 'Basic' category is expanded, showing three properties: 'ID' with value '1', '名称' (Name) with value '課程1' (Course 1), and '学生人数' (Student Count) with value '1'. The interface includes standard window controls and a toolbar with icons for various actions.

Property	Value
Basic	
ID	1
名称	課程1
学生人数	1



This screenshot shows the 'Location' and 'Size' property views in the same software interface. The 'Location' category is expanded, showing 'X' with value '2' and 'Y' with value '10'. The 'Size' category is also expanded, showing 'Height' with value '70' and 'Width' with value '60'. The 'Value' column for 'Location' shows 'Point(2, 10)' and for 'Size' it shows 'Dimension(60, 70)'. The interface includes standard window controls and a toolbar with icons for various actions.

Property	Value
Location	Point(2, 10)
X	2
Y	10
Size	Dimension(60, 70)
Height	70
Width	60

Sample:

Property view for course

